NPH-Trust — Architecture & Design Document

> **Version:** 1.0.0
> **Date:** 2026-04-15
> **Status:** Authoritative design reference — no implementation before this is finalized.
> **Scope:** App 1 (Patient pathway registry / dashboard / provenance / attestation / workflow engine). Schema designed for future App 2 (manuscript generation) consumption.

Table of Contents

1. System Overview
2. Database Schema (Prisma)
3. TypeScript Interfaces
4. Service Boundaries
5. Event Lifecycle States
6. Error Cases & Retry Behavior
7. Chain-Agnostic vs Chain-Specific Separation
8. API Routes
9. Security Model & RBAC
10. Appendix: FHIR Mapping Reference

---

1. System Overview

1.1 Architecture Diagram (Logical)

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Next.js Frontend                            │
│  Dashboard │ Pathway Viewer │ Provenance Inspector │ Admin Panel     │
└──────────────────────────────────────────────────────────────────────┘
                                │ REST API (Next.js API Routes)
┌──────────────────────────────────────────────────────────────────────┐
│                         Service Layer                                 │
│  AuthService │ ProjectService │ PathwayService │ AttestationService  │
│  ProvenanceService │ ImportService │ ExportService │ AuditService    │
│  CheckpointService │ BlockchainService (abstracted)                  │
└──────────────────────────────────────────────────────────────────────┘
                                │ Prisma ORM
┌──────────────────────────────────────────────────────────────────────┐
│                       PostgreSQL Database                             │
│  (Shared between App 1 and future App 2)                              │
└──────────────────────────────────────────────────────────────────────┘
                                │ (optional, async)
┌──────────────────────────────────────────────────────────────────────┐
│                Blockchain Anchoring Layer                             │
│  Chain-agnostic interface → Chain-specific providers                  │
│  (Base L2 default, multi-chain later)                                 │
└──────────────────────────────────────────────────────────────────────┘
```

1.2 Stack

Layer	Technology
Frontend	Next.js 14+ (App Router), TypeScript, Tailwind CSS
API	Next.js API Routes (REST)
ORM	Prisma
Database	PostgreSQL 15+
Auth	Email/password, bcrypt, JWT sessions
Hashing	SHA-256 (node:crypto)
Signing	HMAC-SHA256 (institutional signer, upgradeable to ECDSA)
Blockchain	Ethereum-compatible L2 (Base default), ethers.js

1.3 Key Design Principles

1. **Blockchain is never the source of truth** — PostgreSQL holds all workflow state. Blockchain is an optional anchoring layer for tamper-evidence.
2. **Graceful degradation** — System functions fully without blockchain connectivity.
3. **All PHI off-chain** — Only de-identified abstractions and hashes are stored. No real patient identifiers.
4. **App 2 compatibility** — Schema supports future manuscript generation without migrations.
5. **Deterministic serialization** — All attestation payloads use canonical JSON (sorted keys, no whitespace).
6. **Auditability** — Every state change is logged. Full lineage from input to output.

---

2. Database Schema (Prisma)

2.1 Enums

```
// ═══ Auth & Roles ═══════════════════════════════

enum Role {
  ADMIN
  RESEARCHER
  COORDINATOR
  AUDITOR
}

enum SessionStatus {
  ACTIVE
  EXPIRED
  REVOKED
}

// ═══ Project ═══════════════════════════════

enum ProjectStatus {
  DRAFT
  ACTIVE
  PAUSED
  COMPLETED
  ARCHIVED
}

// ═══ Pathway ═══════════════════════════════

enum PathwayStageType {
  SYMPTOM_SCREENING
  IMAGING
  SPECIALIST_REVIEW
  CSF_TESTING
  TREATMENT_DECISION
  SHUNT_INTERVENTION
  FOLLOW_UP
}

enum PathwayEventStatus {
  PENDING
  IN_PROGRESS
  COMPLETED
  SKIPPED
  CANCELLED
  FAILED
}

// ═══ FHIR Resource Types ═══════════════════

enum FHIRResourceType {
  QUESTIONNAIRE_RESPONSE
  OBSERVATION
  SERVICE_REQUEST
  CARE_PLAN
  DOCUMENT_REFERENCE
}

// ═══ Import ═══════════════════════════════

enum ImportStatus {
  PENDING
  VALIDATING
  VALIDATED
```

```
  IMPORTING
  COMPLETED
  FAILED
  PARTIALLY_COMPLETED
}

enum ImportSourceType {
  CSV
  XLSX
  JSON
  FHIR_BUNDLE
  MANUAL
}

// ── Attestation ──────────────────────────────────────────

enum AttestationStatus {
  DRAFT
  HASHED
  SUBMITTED
  CONFIRMED
  FAILED
  REVERIFIED
}

enum SignatureAlgorithm {
  HMAC_SHA256
  ECDSA_SECP256K1
}

// ── Approval ─────────────────────────────────────────────

enum ApprovalStatus {
  PENDING
  APPROVED
  REJECTED
  WITHDRAWN
}

enum ApprovalTargetType {
  CHECKPOINT
  OUTPUT_ARTIFACT
  PATHWAY_EVENT
  ATTESTATION
}

// ── Artifacts ────────────────────────────────────────────

enum ArtifactStatus {
  PENDING
  PROCESSING
  READY
  ERROR
  ARCHIVED
}

// ── Lock ─────────────────────────────────────────────────

enum LockTargetType {
  PROJECT
  PATIENT_EPISODE
  PATHWAY_EVENT
  OUTPUT_ARTIFACT
```

```
  CHECKPOINT
}
```

2.2 Models

```
// ═══════════════════════════════════════════════════════════════
// AUTH, USERS, SESSIONS
// ═══════════════════════════════════════════════════════════════

model User {
  id            String    @id @default(cuid())
  email         String    @unique
  passwordHash  String    @map("password_hash")
  displayName   String    @map("display_name")
  role          Role      @default(RESEARCHER)
  isActive      Boolean   @default(true) @map("is_active")
  createdAt     DateTime  @default(now()) @map("created_at")
  updatedAt     DateTime  @updatedAt @map("updated_at")

  sessions          Session[]
  projectMembers    ProjectMember[]
  approvals         Approval[]
  locksHeld         Lock[]
  auditEntries      AuditEntry[]       @relation("AuditActor")
  importJobs        ImportJob[]
  attestations      Attestation[]      @relation("AttestationCreator")
  checkpoints       Checkpoint[]       @relation("CheckpointCreator")

  @@map("users")
}

model Session {
  id           String        @id @default(cuid())
  userId       String        @map("user_id")
  token        String        @unique
  status       SessionStatus @default(ACTIVE)
  ipAddress    String?       @map("ip_address")
  userAgent    String?       @map("user_agent")
  expiresAt    DateTime      @map("expires_at")
  createdAt    DateTime      @default(now()) @map("created_at")
  revokedAt    DateTime?     @map("revoked_at")

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([token])
  @@index([userId])
  @@map("sessions")
}

// ═══════════════════════════════════════════════════════════════
// PROJECTS & SITES
// ═══════════════════════════════════════════════════════════════

model Project {
  id            String        @id @default(cuid())
  name          String
  description   String?
  status        ProjectStatus @default(DRAFT)
  metadata      Json?         // extensible key-value (protocol info, study identifi-
ers)
  createdAt     DateTime      @default(now()) @map("created_at")
  updatedAt     DateTime      @updatedAt @map("updated_at")

  sites             Site[]
  members           ProjectMember[]
  patientEpisodes   PatientEpisode[]
  inputArtifacts    InputArtifact[]
```

```
  outputArtifacts   OutputArtifact[]
  attestations      Attestation[]
  checkpoints       Checkpoint[]
  importJobs        ImportJob[]
  runLogs           RunLog[]

  @@map("projects")
}

model Site {
  id            String   @id @default(cuid())
  projectId     String   @map("project_id")
  name          String
  identifier    String   // e.g. site code
  metadata      Json?    // site configuration metadata
  isActive      Boolean  @default(true) @map("is_active")
  createdAt     DateTime @default(now()) @map("created_at")
  updatedAt     DateTime @updatedAt @map("updated_at")

  project         Project          @relation(fields: [projectId], references: [id], on
Delete: Cascade)
  patientEpisodes PatientEpisode[]

  @@unique([projectId, identifier])
  @@map("sites")
}

model ProjectMember {
  id        String   @id @default(cuid())
  projectId String   @map("project_id")
  userId    String   @map("user_id")
  role      Role     // role within this project (may differ from global role)
  joinedAt  DateTime @default(now()) @map("joined_at")

  project Project @relation(fields: [projectId], references: [id], onDelete: Cascade)
  user    User    @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([projectId, userId])
  @@map("project_members")
}

// ═══════════════════════════════════════════════════════════════
// PATIENT EPISODES (de-identified abstractions)
// ═══════════════════════════════════════════════════════════════

model PatientEpisode {
  id              String   @id @default(cuid())
  projectId       String   @map("project_id")
  siteId          String?  @map("site_id")
  pseudoId        String   @map("pseudo_id")  // de-identified patient identifier
  metadata        Json?    // demographics bucket (age range, sex — never DOB or name)
  createdAt       DateTime @default(now()) @map("created_at")
  updatedAt       DateTime @updatedAt @map("updated_at")

  project       Project        @relation(fields: [projectId], references: [id], onDele
te: Cascade)
  site          Site?          @relation(fields: [siteId], references: [id])
  pathwayEvents PathwayEvent[]
  fhirResources FHIRResource[]

  @@unique([projectId, pseudoId])
  @@index([projectId])
  @@map("patient_episodes")
```

```
}

// ═══════════════════════════════════════════════════════════════
// PATHWAY STAGES & EVENTS
// ═══════════════════════════════════════════════════════════════

model PathwayStageDefinition {
  id          String           @id @default(cuid())
  stageType   PathwayStageType @map("stage_type")
  name        String
  description String?
  sortOrder   Int              @map("sort_order")
  metadata    Json?            // expected observations, questionnaires, etc.

  events PathwayEvent[]

  @@unique([stageType])
  @@map("pathway_stage_definitions")
}

model PathwayEvent {
  id                  String             @id @default(cuid())
  patientEpisodeId    String             @map("patient_episode_id")
  stageDefinitionId   String             @map("stage_definition_id")
  status              PathwayEventStatus @default(PENDING)
  occurredAt          DateTime?          @map("occurred_at")
  completedAt         DateTime?          @map("completed_at")
  performedBy         String?            @map("performed_by") // reference descrip-
tion, not userId
  notes               String?
  data                Json?              // stage-specific structured data
  metadata            Json?
  createdAt           DateTime           @default(now()) @map("created_at")
  updatedAt           DateTime           @updatedAt @map("updated_at")

  patientEpisode  PatientEpisode       @relation(fields: [patientEpisodeId], refer-
ences: [id], onDelete: Cascade)
  stageDefinition PathwayStageDefinition @relation(fields: [stageDefinitionId], refer-
ences: [id])
  attestations    Attestation[]        @relation("PathwayEventAttestations")
  fhirResources   FHIRResource[]
  approvals       Approval[]           @relation("PathwayEventApprovals")

  @@index([patientEpisodeId])
  @@index([stageDefinitionId])
  @@map("pathway_events")
}

// ═══════════════════════════════════════════════════════════════
// FHIR-ALIGNED RESOURCES
// ═══════════════════════════════════════════════════════════════

model FHIRResource {
  id                 String           @id @default(cuid())
  resourceType       FHIRResourceType @map("resource_type")
  patientEpisodeId   String           @map("patient_episode_id")
  pathwayEventId     String?          @map("pathway_event_id")
  resourceId         String           @map("resource_id")  // FHIR-style logical id
  data               Json             // the FHIR-aligned resource payload
  version            Int              @default(1)
  isActive           Boolean          @default(true) @map("is_active")
  createdAt          DateTime         @default(now()) @map("created_at")
  updatedAt          DateTime         @updatedAt @map("updated_at")
```

```
  patientEpisode PatientEpisode @relation(fields: [patientEpisodeId], references: [id]
, onDelete: Cascade)
  pathwayEvent   PathwayEvent?  @relation(fields: [pathwayEventId], references: [id])
  lineageEntries LineageEntry[] @relation("FHIRResourceLineage")

  @@unique([resourceType, resourceId, version])
  @@index([patientEpisodeId])
  @@index([resourceType])
  @@map("fhir_resources")
}

// ═══════════════════════════════════════════════════════════════
// INPUT ARTIFACTS
// ═══════════════════════════════════════════════════════════════

model InputArtifact {
  id              String         @id @default(cuid())
  projectId       String         @map("project_id")
  filename        String
  mimeType        String         @map("mime_type")
  sizeBytes       Int            @map("size_bytes")
  storagePath     String         @map("storage_path")  // filesystem or object-
storage path
  sha256Hash      String         @map("sha256_hash")
  sourceType      ImportSourceType @map("source_type")
  metadata        Json?
  status          ArtifactStatus @default(PENDING)
  uploadedAt      DateTime       @default(now()) @map("uploaded_at")
  createdAt       DateTime       @default(now()) @map("created_at")

  project        Project        @relation(fields: [projectId], references: [id], onDel
ete: Cascade)
  importJobs     ImportJob[]
  lineageEntries LineageEntry[] @relation("InputArtifactLineage")

  @@index([projectId])
  @@map("input_artifacts")
}

// ═══════════════════════════════════════════════════════════════
// IMPORT JOBS
// ═══════════════════════════════════════════════════════════════

model ImportJob {
  id              String       @id @default(cuid())
  projectId       String       @map("project_id")
  inputArtifactId String       @map("input_artifact_id")
  initiatedById   String       @map("initiated_by_id")
  status          ImportStatus @default(PENDING)
  sourceType      ImportSourceType @map("source_type")
  totalRows       Int?         @map("total_rows")
  processedRows   Int?         @default(0) @map("processed_rows")
  errorRows       Int?         @default(0) @map("error_rows")
  errors          Json?        // array of { row, field, message }
  startedAt       DateTime?    @map("started_at")
  completedAt     DateTime?    @map("completed_at")
  createdAt       DateTime     @default(now()) @map("created_at")

  project       Project       @relation(fields: [projectId], references: [id], onDelet
e: Cascade)
  inputArtifact InputArtifact @relation(fields: [inputArtifactId], references: [id])
  initiatedBy   User          @relation(fields: [initiatedById], references: [id])
```

```
  @@index([projectId])
  @@map("import_jobs")
}

// ═══════════════════════════════════════════════════════════
// ATTESTATION & BLOCKCHAIN ANCHORING
// ═══════════════════════════════════════════════════════════

model Attestation {
  id                 String             @id @default(cuid())
  projectId          String             @map("project_id")
  pathwayEventId     String?            @map("pathway_event_id")
  createdById        String             @map("created_by_id")

  // Payload
  payloadCanonical   String             @map("payload_canonical") @db.Text  // canonic-
al JSON
  payloadHash        String             @map("payload_hash")       // SHA-256 hex

  // Signature
  signatureAlgo      SignatureAlgorithm @default(HMAC_SHA256) @map("signature_algo")
  signature          String             // hex-encoded
  signerId           String             @map("signer_id")          // institutional sig
ner identifier

  // Lifecycle
  status             AttestationStatus  @default(DRAFT)

  // Blockchain anchor (nullable — may never be anchored)
  anchorChainId      String?            @map("anchor_chain_id")    // e.g. "base", "op-
timism"
  anchorTxHash       String?            @map("anchor_tx_hash")
  anchorBlockNumber  Int?               @map("anchor_block_number")
  anchorTimestamp    DateTime?          @map("anchor_timestamp")
  anchorContractAddr String?            @map("anchor_contract_addr")
  anchorError        String?            @map("anchor_error")

  // Verification
  lastVerifiedAt     DateTime?          @map("last_verified_at")
  verificationNote   String?            @map("verification_note")

  createdAt          DateTime           @default(now()) @map("created_at")
  updatedAt          DateTime           @updatedAt @map("updated_at")

  project      Project       @relation(fields: [projectId], references: [id], onDelete
: Cascade)
  pathwayEvent PathwayEvent? @relation("PathwayEventAttestations", fields: [path-
wayEventId], references: [id])
  createdBy    User          @relation("AttestationCreator", fields: [createdById], re
ferences: [id])

  @@index([projectId])
  @@index([payloadHash])
  @@index([status])
  @@map("attestations")
}

// ═══════════════════════════════════════════════════════════
// PROVENANCE & LINEAGE
// ═══════════════════════════════════════════════════════════

model LineageEntry {
```

```
  id                 String   @id @default(cuid())
  projectId          String   @map("project_id")

  // Source
  sourceType         String   @map("source_type")  // "input_artifact" | "fhir_resource
" | "pathway_event" | "run_log"
  sourceId           String   @map("source_id")

  // Target
  targetType         String   @map("target_type")  // "fhir_resource" |
"pathway_event" | "output_artifact" | "attestation"
  targetId           String   @map("target_id")

  transformDesc      String?  @map("transform_desc") // human-readable description of t
ransformation
  metadata           Json?
  createdAt          DateTime @default(now()) @map("created_at")

  // Optional typed relations for common cases
  inputArtifact      InputArtifact?  @relation("InputArtifactLineage", fields: [sourceI
d], references: [id], map: "lineage_input_artifact_fk")
  fhirResource       FHIRResource?   @relation("FHIRResourceLineage", fields: [target-
Id], references: [id], map: "lineage_fhir_resource_fk")

  @@index([sourceType, sourceId])
  @@index([targetType, targetId])
  @@index([projectId])
  @@map("lineage_entries")
}

// ═══════════════════════════════════════════════════════════
// EXECUTION HISTORY / RUN LOG
// ═══════════════════════════════════════════════════════════

model RunLog {
  id             String   @id @default(cuid())
  projectId      String   @map("project_id")
  action         String   // e.g. "import", "attest", "export", "pathway_advance"
  status         String   // "started" | "completed" | "failed"
  inputSummary   Json?    @map("input_summary")
  outputSummary  Json?    @map("output_summary")
  errorDetail    String?  @map("error_detail") @db.Text
  durationMs     Int?     @map("duration_ms")
  triggeredBy    String?  @map("triggered_by") // userId or "system"
  createdAt      DateTime @default(now()) @map("created_at")

  project Project @relation(fields: [projectId], references: [id], onDelete: Cascade)

  @@index([projectId])
  @@index([action])
  @@map("run_logs")
}

// ═══════════════════════════════════════════════════════════
// OUTPUT ARTIFACTS
// ═══════════════════════════════════════════════════════════

model OutputArtifact {
  id             String         @id @default(cuid())
  projectId      String         @map("project_id")
  artifactType   String         @map("artifact_type")  // "dashboard_export", "fhir_bun
dle", "csv_report", "manuscript_draft"
  filename       String
```

```
  mimeType      String         @map("mime_type")
  sizeBytes     Int?           @map("size_bytes")
  storagePath   String         @map("storage_path")
  sha256Hash    String         @map("sha256_hash")
  status        ArtifactStatus @default(PENDING)
  metadata      Json?
  createdAt     DateTime       @default(now()) @map("created_at")
  updatedAt     DateTime       @updatedAt @map("updated_at")

  project   Project    @relation(fields: [projectId], references: [id], onDelete: Cas-
cade)
  approvals Approval[] @relation("OutputArtifactApprovals")

  @@index([projectId])
  @@map("output_artifacts")
}

// ═══════════════════════════════════════════════════════════
// APPROVALS
// ═══════════════════════════════════════════════════════════

model Approval {
  id              String             @id @default(cuid())
  targetType      ApprovalTargetType @map("target_type")
  targetId        String             @map("target_id")
  requestedById   String?            @map("requested_by_id")
  reviewedById    String?            @map("reviewed_by_id")
  status          ApprovalStatus     @default(PENDING)
  comment         String?            @db.Text
  requestedAt     DateTime           @default(now()) @map("requested_at")
  reviewedAt      DateTime?          @map("reviewed_at")

  reviewedBy      User?           @relation(fields: [reviewedById], references: [id])
  pathwayEvent    PathwayEvent?   @relation("PathwayEventApprovals", fields: [target-
Id], references: [id], map: "approval_pathway_event_fk")
  outputArtifact  OutputArtifact? @relation("OutputArtifactApprovals", fields: [tar-
getId], references: [id], map: "approval_output_artifact_fk")

  @@index([targetType, targetId])
  @@index([status])
  @@map("approvals")
}

// ═══════════════════════════════════════════════════════════
// LOCKS
// ═══════════════════════════════════════════════════════════

model Lock {
  id          String         @id @default(cuid())
  targetType  LockTargetType @map("target_type")
  targetId    String         @map("target_id")
  heldById    String         @map("held_by_id")
  reason      String?
  acquiredAt  DateTime       @default(now()) @map("acquired_at")
  expiresAt   DateTime?      @map("expires_at")

  heldBy User @relation(fields: [heldById], references: [id])

  @@unique([targetType, targetId])
  @@map("locks")
}

// ═══════════════════════════════════════════════════════════
```

```
// CHECKPOINTS / VERSIONS
// ═══════════════════════════════════════════════════════════════

model Checkpoint {
  id            String   @id @default(cuid())
  projectId     String   @map("project_id")
  version       Int
  label         String?
  description   String?  @db.Text
  createdById   String   @map("created_by_id")
  snapshotData  Json     @map("snapshot_data") // serialized state snapshot
  sha256Hash    String   @map("sha256_hash")   // hash of snapshotData
  parentId      String?  @map("parent_id")
  createdAt     DateTime @default(now()) @map("created_at")

  project   Project     @relation(fields: [projectId], references: [id], onDelete: Cas
cade)
  createdBy User        @relation("CheckpointCreator", fields: [createdById], refer-
ences: [id])
  parent    Checkpoint? @relation("CheckpointChain", fields: [parentId], references: [
id])
  children  Checkpoint[] @relation("CheckpointChain")

  @@unique([projectId, version])
  @@index([projectId])
  @@map("checkpoints")
}

// ═══════════════════════════════════════════════════════════════
// AUDIT LOG
// ═══════════════════════════════════════════════════════════════

model AuditEntry {
  id          String   @id @default(cuid())
  actorId     String?  @map("actor_id")
  action      String   // e.g. "user.login", "project.create", "attestation.submit"
  entityType  String   @map("entity_type")
  entityId    String   @map("entity_id")
  before      Json?    // previous state (for updates)
  after       Json?    // new state (for updates)
  ipAddress   String?  @map("ip_address")
  metadata    Json?
  createdAt   DateTime @default(now()) @map("created_at")

  actor User? @relation("AuditActor", fields: [actorId], references: [id])

  @@index([entityType, entityId])
  @@index([actorId])
  @@index([action])
  @@index([createdAt])
  @@map("audit_entries")
}
```

2.3 Design Notes for App 2 Consumption

App 2 (manuscript generation) will read from the shared database using these tables:

App 2 Need	Source Table(s)
Project metadata & study info	`projects`, `sites`, `project_members`
Patient cohort data	`patient_episodes`, `fhir_resources`
Pathway completion status	`pathway_events`, `pathway_stage_definitions`
Evidence provenance	`attestations`, `lineage_entries`
Generated artifacts	`output_artifacts`
Version history	`checkpoints`
Approval state	`approvals`
Run history	`run_logs`

App 2 should use **read-only** access to App 1 tables and write only to its own `output_artifacts` entries (with `artifact_type = 'manuscript_draft'`).

---

3. TypeScript Interfaces

3.1 Core Identity & Auth

```typescript
// ─── Roles & Auth ──────────────────────────────────────────

type Role = 'ADMIN' | 'RESEARCHER' | 'COORDINATOR' | 'AUDITOR';

interface User {
  id: string;
  email: string;
  displayName: string;
  role: Role;
  isActive: boolean;
  createdAt: Date;
  updatedAt: Date;
}

interface Session {
  id: string;
  userId: string;
  token: string;
  status: 'ACTIVE' | 'EXPIRED' | 'REVOKED';
  ipAddress?: string;
  userAgent?: string;
  expiresAt: Date;
  createdAt: Date;
}

interface AuthContext {
  user: User;
  session: Session;
  projectRole?: Role; // role within current project context
}
```

3.2 Project & Site

```typescript
type ProjectStatus = 'DRAFT' | 'ACTIVE' | 'PAUSED' | 'COMPLETED' | 'ARCHIVED';

interface Project {
  id: string;
  name: string;
  description?: string;
  status: ProjectStatus;
  metadata?: Record<string, unknown>;
  createdAt: Date;
  updatedAt: Date;
}

interface Site {
  id: string;
  projectId: string;
  name: string;
  identifier: string;
  metadata?: Record<string, unknown>;
  isActive: boolean;
  createdAt: Date;
  updatedAt: Date;
}

interface ProjectMember {
  id: string;
  projectId: string;
  userId: string;
  role: Role;
  joinedAt: Date;
}
```

3.3 Patient Episodes

```typescript
interface PatientEpisode {
  id: string;
  projectId: string;
  siteId?: string;
  pseudoId: string;            // de-identified identifier
  metadata?: {
    ageRange?: string;         // e.g. "70-79"
    sex?: 'M' | 'F' | 'OTHER' | 'UNKNOWN';
    enrollmentDate?: string;   // ISO date
    [key: string]: unknown;
  };
  createdAt: Date;
  updatedAt: Date;
}
```

3.4 Pathway

```typescript
type PathwayStageType =
  | 'SYMPTOM_SCREENING'
  | 'IMAGING'
  | 'SPECIALIST_REVIEW'
  | 'CSF_TESTING'
  | 'TREATMENT_DECISION'
  | 'SHUNT_INTERVENTION'
  | 'FOLLOW_UP';

type PathwayEventStatus =
  | 'PENDING'
  | 'IN_PROGRESS'
  | 'COMPLETED'
  | 'SKIPPED'
  | 'CANCELLED'
  | 'FAILED';

interface PathwayStage {
  id: string;
  stageType: PathwayStageType;
  name: string;
  description?: string;
  sortOrder: number;
  metadata?: Record<string, unknown>;
}

interface PathwayEvent {
  id: string;
  patientEpisodeId: string;
  stageDefinitionId: string;
  status: PathwayEventStatus;
  occurredAt?: Date;
  completedAt?: Date;
  performedBy?: string;
  notes?: string;
  data?: Record<string, unknown>;  // stage-specific structured data
  metadata?: Record<string, unknown>;
  createdAt: Date;
  updatedAt: Date;
}
```

3.5 FHIR-Aligned Resources

```typescript
type FHIRResourceType =
  | 'QUESTIONNAIRE_RESPONSE'
  | 'OBSERVATION'
  | 'SERVICE_REQUEST'
  | 'CARE_PLAN'
  | 'DOCUMENT_REFERENCE';

// ─── Base ──────────────────────────────────────────────────

interface FHIRResource<T extends FHIRResourceType = FHIRResourceType> {
  id: string;
  resourceType: T;
  patientEpisodeId: string;
  pathwayEventId?: string;
  resourceId: string;         // FHIR-style logical ID
  data: FHIRResourceData<T>;
  version: number;
  isActive: boolean;
  createdAt: Date;
  updatedAt: Date;
}

// ─── Type-specific data payloads ──────────────────────────

type FHIRResourceData<T extends FHIRResourceType> =
  T extends 'QUESTIONNAIRE_RESPONSE' ? FHIRQuestionnaireResponseData :
  T extends 'OBSERVATION' ? FHIRObservationData :
  T extends 'SERVICE_REQUEST' ? FHIRServiceRequestData :
  T extends 'CARE_PLAN' ? FHIRCarePlanData :
  T extends 'DOCUMENT_REFERENCE' ? FHIRDocumentReferenceData :
  Record<string, unknown>;

interface FHIRQuestionnaireResponseData {
  questionnaireId: string;        // which questionnaire (e.g. "inph-screening-v1")
  status: 'in-progress' | 'completed' | 'amended';
  authored?: string;              // ISO datetime
  items: FHIRQuestionnaireItem[];
}

interface FHIRQuestionnaireItem {
  linkId: string;
  text: string;
  answer: Array<{
    valueString?: string;
    valueInteger?: number;
    valueDecimal?: number;
    valueBoolean?: boolean;
    valueCoding?: { system: string; code: string; display: string };
    valueDate?: string;
  }>;
}

interface FHIRObservationData {
  code: { system: string; code: string; display: string };
  status: 'preliminary' | 'final' | 'amended' | 'cancelled';
  effectiveDateTime?: string;
  valueQuantity?: { value: number; unit: string; system?: string; code?: string };
  valueString?: string;
  valueCodeableConcept?: { system: string; code: string; display: string };
  interpretation?: { system: string; code: string; display: string };
  referenceRange?: { low?: number; high?: number; unit?: string };
  component?: Array<{
```

```
    code: { system: string; code: string; display: string };
    valueQuantity?: { value: number; unit: string };
    valueString?: string;
  }>;
}

interface FHIRServiceRequestData {
  status: 'draft' | 'active' | 'completed' | 'revoked';
  intent: 'proposal' | 'plan' | 'order';
  code: { system: string; code: string; display: string };
  reasonCode?: Array<{ system: string; code: string; display: string }>;
  authoredOn?: string;
  note?: string;
}

interface FHIRCarePlanData {
  status: 'draft' | 'active' | 'completed' | 'revoked';
  intent: 'proposal' | 'plan' | 'order';
  title: string;
  description?: string;
  period?: { start: string; end?: string };
  activities: Array<{
    detail: {
      status: string;
      code: { system: string; code: string; display: string };
      description?: string;
      scheduledString?: string;
    };
  }>;
}

interface FHIRDocumentReferenceData {
  status: 'current' | 'superseded';
  type: { system: string; code: string; display: string };
  description?: string;
  date?: string;
  content: Array<{
    attachment: {
      contentType: string;
      url?: string;                // internal reference to storage
      title?: string;
      size?: number;
      hash?: string;              // SHA-256
    };
  }>;
}
```

3.6 Attestation & Blockchain

```
type AttestationStatus =
  | 'DRAFT'
  | 'HASHED'
  | 'SUBMITTED'
  | 'CONFIRMED'
  | 'FAILED'
  | 'REVERIFIED';

type SignatureAlgorithm = 'HMAC_SHA256' | 'ECDSA_SECP256K1';

// ─── Attestation Payload ──────────────────────────────────

interface AttestationPayload {
  /** Unique attestation identifier */
  attestationId: string;
  /** What is being attested */
  subjectType: string;           // "pathway_event" | "checkpoint" | "output_artifact"
  subjectId: string;
  /** Project context */
  projectId: string;
  /** Canonical data snapshot (deterministically serialized) */
  canonicalData: Record<string, unknown>;
  /** ISO timestamp of attestation creation */
  timestamp: string;
  /** Version of the payload schema */
  schemaVersion: string;
}

// ─── Anchor Record ────────────────────────────────────────

interface AnchorRecord {
  chainId: string;               // "base", "optimism", "polygon", etc.
  txHash: string;
  blockNumber: number;
  timestamp: Date;
  contractAddress: string;
  payloadHash: string;           // the hash that was anchored
  error?: string;
}

// ─── Verification ─────────────────────────────────────────

interface VerificationResult {
  attestationId: string;
  payloadHash: string;
  /** Does the current canonical payload still hash to the stored hash? */
  payloadIntegrity: boolean;
  /** Does the stored signature verify against the hash? */
  signatureValid: boolean;
  /** Was the hash found on-chain (if anchored)? */
  anchorVerified: boolean | null; // null if never anchored
  /** Anchor details if verified */
  anchorRecord?: AnchorRecord;
  /** Overall status */
  status: 'VERIFIED' | 'SIGNATURE_MISMATCH' | 'PAYLOAD_TAMPERED' | 'ANCHOR_MISSING' |
'ANCHOR_MISMATCH' | 'CHAIN_UNAVAILABLE';
  verifiedAt: Date;
  notes?: string;
}

// ─── Blockchain Provider (chain-agnostic interface) ───────
```

```typescript
interface BlockchainProvider {
  /** Provider identifier (e.g. "base", "optimism") */
  readonly chainId: string;

  /** Submit a hash to be anchored on-chain */
  submitAnchor(payloadHash: string): Promise<AnchorSubmission>;

  /** Verify that a hash exists on-chain */
  verifyAnchor(payloadHash: string, txHash: string): Promise<AnchorVerification>;

  /** Get the status of a previously submitted anchor */
  getAnchorStatus(txHash: string): Promise<AnchorStatus>;

  /** Check if the provider is available */
  isAvailable(): Promise<boolean>;
}

interface AnchorSubmission {
  success: boolean;
  txHash?: string;
  error?: string;
}

interface AnchorVerification {
  found: boolean;
  hashMatch: boolean;
  blockNumber?: number;
  timestamp?: Date;
  error?: string;
}

interface AnchorStatus {
  txHash: string;
  status: 'PENDING' | 'CONFIRMED' | 'FAILED' | 'NOT_FOUND';
  blockNumber?: number;
  confirmations?: number;
  error?: string;
}
```

3.7 Artifacts

```typescript
type ArtifactStatus = 'PENDING' | 'PROCESSING' | 'READY' | 'ERROR' | 'ARCHIVED';
type ImportSourceType = 'CSV' | 'XLSX' | 'JSON' | 'FHIR_BUNDLE' | 'MANUAL';

interface InputArtifact {
  id: string;
  projectId: string;
  filename: string;
  mimeType: string;
  sizeBytes: number;
  storagePath: string;
  sha256Hash: string;
  sourceType: ImportSourceType;
  metadata?: Record<string, unknown>;
  status: ArtifactStatus;
  uploadedAt: Date;
  createdAt: Date;
}

interface OutputArtifact {
  id: string;
  projectId: string;
  artifactType: string;
  filename: string;
  mimeType: string;
  sizeBytes?: number;
  storagePath: string;
  sha256Hash: string;
  status: ArtifactStatus;
  metadata?: Record<string, unknown>;
  createdAt: Date;
  updatedAt: Date;
}
```

3.8 Checkpoints, Approvals, Locks

```
interface Checkpoint {
  id: string;
  projectId: string;
  version: number;
  label?: string;
  description?: string;
  createdById: string;
  snapshotData: Record<string, unknown>;
  sha256Hash: string;
  parentId?: string;
  createdAt: Date;
}

type ApprovalStatus = 'PENDING' | 'APPROVED' | 'REJECTED' | 'WITHDRAWN';
type ApprovalTargetType = 'CHECKPOINT' | 'OUTPUT_ARTIFACT' | 'PATHWAY_EVENT' | 'AT-
TESTATION';

interface Approval {
  id: string;
  targetType: ApprovalTargetType;
  targetId: string;
  requestedById?: string;
  reviewedById?: string;
  status: ApprovalStatus;
  comment?: string;
  requestedAt: Date;
  reviewedAt?: Date;
}

type LockTargetType = 'PROJECT' | 'PATIENT_EPISODE' | 'PATHWAY_EVENT' | 'OUT-
PUT_ARTIFACT' | 'CHECKPOINT';

interface Lock {
  id: string;
  targetType: LockTargetType;
  targetId: string;
  heldById: string;
  reason?: string;
  acquiredAt: Date;
  expiresAt?: Date;
}
```

3.9 Provenance & Lineage

```
interface LineageEntry {
  id: string;
  projectId: string;
  sourceType: 'input_artifact' | 'fhir_resource' | 'pathway_event' | 'run_log';
  sourceId: string;
  targetType: 'fhir_resource' | 'pathway_event' | 'output_artifact' | 'attestation';
  targetId: string;
  transformDesc?: string;
  metadata?: Record<string, unknown>;
  createdAt: Date;
}

/** Full provenance chain for the Provenance Inspector UI */
interface ProvenanceRecord {
  /** The entity being inspected */
  entityType: string;
  entityId: string;
  /** Direct lineage */
  upstream: LineageEntry[];    // what produced this entity
  downstream: LineageEntry[];  // what this entity feeds into
  /** Related attestations */
  attestations: Array<{
    id: string;
    status: AttestationStatus;
    payloadHash: string;
    anchorRecord?: AnchorRecord;
  }>;
  /** Full hash and integrity info */
  currentHash?: string;
  integrityVerified?: boolean;
}
```

3.10 Audit & Run Log

```
interface AuditEntry {
  id: string;
  actorId?: string;
  action: string;
  entityType: string;
  entityId: string;
  before?: Record<string, unknown>;
  after?: Record<string, unknown>;
  ipAddress?: string;
  metadata?: Record<string, unknown>;
  createdAt: Date;
}

interface RunLog {
  id: string;
  projectId: string;
  action: string;
  status: 'started' | 'completed' | 'failed';
  inputSummary?: Record<string, unknown>;
  outputSummary?: Record<string, unknown>;
  errorDetail?: string;
  durationMs?: number;
  triggeredBy?: string;
  createdAt: Date;
}
```

---

4. Service Boundaries

Each service is a module in `src/lib/services/`. Services are stateless and receive dependencies via function parameters (Prisma client, auth context, etc.).

4.1 AuthService

```
src/lib/services/auth.service.ts

Responsibilities:
  - User registration and login (email/password)
  - Password hashing (bcrypt)
  - Session creation and validation (JWT)
  - Session revocation
  - Role checking helpers

Dependencies: Prisma, bcrypt, jsonwebtoken

Key Functions:
  register(email, password, displayName, role?) → User
  login(email, password) → { user, session, token }
  validateSession(token) → AuthContext | null
  revokeSession(sessionId) → void
  changePassword(userId, oldPassword, newPassword) → void
  hasRole(authContext, requiredRole) → boolean
  hasProjectRole(authContext, projectId, requiredRole) → boolean
```

4.2 ProjectService

```
src/lib/services/project.service.ts

Responsibilities:
  - CRUD for projects
  - Site management within projects
  - Member management and role assignment
  - Project status transitions

Dependencies: Prisma, AuthService, AuditService

Key Functions:
  createProject(auth, data) → Project
  updateProject(auth, projectId, data) → Project
  setProjectStatus(auth, projectId, status) → Project
  addSite(auth, projectId, siteData) → Site
  updateSite(auth, siteId, data) → Site
  addMember(auth, projectId, userId, role) → ProjectMember
  removeMember(auth, projectId, userId) → void
  getProjectWithDetails(projectId) → ProjectWithRelations
  listProjects(auth, filters?) → Project[]
```

4.3 PathwayService

```
src/lib/services/pathway.service.ts

Responsibilities:
  - Patient episode CRUD
  - Pathway event creation and progression
  - Stage definition management
  - Enforcing valid stage transitions
  - Linking FHIR resources to pathway events

Dependencies: Prisma, AuditService, AttestationService (optional trigger)

Key Functions:
  createPatientEpisode(auth, projectId, data) → PatientEpisode
  updatePatientEpisode(auth, episodeId, data) → PatientEpisode
  createPathwayEvent(auth, episodeId, stageType, data) → PathwayEvent
  updatePathwayEventStatus(auth, eventId, status) → PathwayEvent
  getEpisodeTimeline(episodeId) → PathwayEvent[]
  getPathwayProgress(episodeId) → PathwayProgressSummary
  listStageDefinitions() → PathwayStage[]
  linkFHIRResource(eventId, resourceId) → void
```

4.4 AttestationService

```
src/lib/services/attestation.service.ts

Responsibilities:
  - Deterministic payload serialization (canonical JSON)
  - SHA-256 hashing
  - HMAC-SHA256 signing
  - Attestation lifecycle management
  - Verification (payload integrity + signature + optional chain)
  - Delegation to BlockchainService for anchoring

Dependencies: Prisma, crypto, BlockchainService, AuditService

Key Functions:
  createAttestation(auth, projectId, subjectType, subjectId, canonicalData) → Attesta-
tion
  hashAttestation(attestationId) → Attestation  // DRAFT → HASHED
  signAttestation(attestationId) → Attestation   // signs with institutional key
  submitAnchor(attestationId) → Attestation       // HASHED → SUBMITTED
  verifyAttestation(attestationId) → VerificationResult
  reverify(attestationId) → VerificationResult
  getAttestation(attestationId) → Attestation
  listAttestations(projectId, filters?) → Attestation[]

Internal (private):
  canonicalize(payload: AttestationPayload) → string
  computeHash(canonical: string) → string
  sign(hash: string) → { signature: string, signerId: string, algorithm: SignatureAl-
gorithm }
```

4.5 ProvenanceService

```
src/lib/services/provenance.service.ts

Responsibilities:
  - Lineage entry creation
  - Full provenance chain retrieval
  - Provenance record assembly for the inspector UI
  - Integrity checks across the lineage chain

Dependencies: Prisma, AttestationService

Key Functions:
  recordLineage(projectId, source, target, transformDesc?) → LineageEntry
  getProvenance(entityType, entityId) → ProvenanceRecord
  getFullChain(entityType, entityId) → LineageEntry[]  // recursive upstream
  verifyChainIntegrity(entityType, entityId) → IntegrityReport
```

4.6 ImportService

```
src/lib/services/import.service.ts

Responsibilities:
  - File upload and InputArtifact creation
  - File validation (format, schema, completeness)
  - Data parsing (CSV, XLSX, JSON, FHIR bundle)
  - Row-level error tracking
  - Creating PatientEpisodes, PathwayEvents, FHIRResources from imported data
  - Import job lifecycle management

Dependencies: Prisma, PathwayService, AuditService, ProvenanceService

Key Functions:
  uploadFile(auth, projectId, file, sourceType) → InputArtifact
  startImport(auth, artifactId) → ImportJob
  validateImport(jobId) → ValidationResult
  executeImport(jobId) → ImportJob
  getImportStatus(jobId) → ImportJob
  listImports(projectId) → ImportJob[]

Internal (private):
  parseCSV(buffer) → ParsedRow[]
  parseXLSX(buffer) → ParsedRow[]
  parseFHIRBundle(json) → FHIRResource[]
  validateRow(row, schema) → ValidationError[]
```

4.7 ExportService

```
src/lib/services/export.service.ts

Responsibilities:
  - Generate output artifacts (FHIR bundles, CSV reports, dashboard exports)
  - Package project data for download
  - Hash and register output artifacts
  - Link output artifacts to provenance chain

Dependencies: Prisma, ProvenanceService, AttestationService

Key Functions:
  exportFHIRBundle(auth, projectId, filters?) → OutputArtifact
  exportCSVReport(auth, projectId, reportType) → OutputArtifact
  exportProjectPackage(auth, projectId) → OutputArtifact
  getOutputArtifact(artifactId) → OutputArtifact
  listOutputArtifacts(projectId) → OutputArtifact[]
```

4.8 BlockchainService

```
src/lib/services/blockchain.service.ts

Responsibilities:
  - Chain-agnostic orchestration layer
  - Provider registration and selection
  - Delegating submitAnchor / verifyAnchor / getStatus to the active provider
  - Graceful degradation when no provider is available

Dependencies: BlockchainProvider implementations

Key Functions:
  registerProvider(provider: BlockchainProvider) → void
  getProvider(chainId?: string) → BlockchainProvider | null
  isAvailable(chainId?: string) → Promise<boolean>
  submitAnchor(payloadHash: string, chainId?: string) → Promise<AnchorSubmission>
  verifyAnchor(payloadHash: string, txHash: string, chainId?: string) → Promise<An-
chorVerification>
  getAnchorStatus(txHash: string, chainId?: string) → Promise<AnchorStatus>

Provider Implementations (separate files):
  src/lib/services/blockchain/base.provider.ts     → BaseBlockchainProvider
implements BlockchainProvider
  src/lib/services/blockchain/noop.provider.ts     → NoopBlockchainProvider (always re
turns success, for testing)
```

4.9 CheckpointService

```
src/lib/services/checkpoint.service.ts

Responsibilities:
  - Create project snapshots (version checkpoints)
  - Serialize project state deterministically
  - Hash snapshot for integrity
  - Checkpoint comparison (diff)
  - Restore/rollback support (metadata only, not destructive)

Dependencies: Prisma, AttestationService, AuditService

Key Functions:
  createCheckpoint(auth, projectId, label?) → Checkpoint
  getCheckpoint(checkpointId) → Checkpoint
  listCheckpoints(projectId) → Checkpoint[]
  compareCheckpoints(checkpointId1, checkpointId2) → CheckpointDiff
  getLatestCheckpoint(projectId) → Checkpoint | null
```

4.10 AuditService

```
src/lib/services/audit.service.ts

Responsibilities:
  - Record all state-changing operations
  - Before/after state capture
  - Query audit trail by entity, actor, time range
  - Run log management

Dependencies: Prisma

Key Functions:
  log(actorId, action, entityType, entityId, before?, after?, metadata?) → AuditEntry
  logRun(projectId, action, triggeredBy, fn) → RunLog  // wraps execution and records
duration/result
  getAuditTrail(entityType, entityId) → AuditEntry[]
  getAuditTrailByActor(actorId, filters?) → AuditEntry[]
  getRunLogs(projectId, filters?) → RunLog[]
  searchAudit(query) → AuditEntry[]
```

---

5. Event Lifecycle States

5.1 Attestation Lifecycle

```
              ┌──────────────┐
              │   DRAFT      │  Payload assembled, not yet hashed
              └──────────────┘
                    │ hashAttestation()
                    ▼
              ┌──────────────┐
              │   HASHED     │  SHA-256 computed, signature created
              └──────────────┘
                    │ submitAnchor()
                    ▼
              ┌──────────────┐
              │ SUBMITTED    │  Tx broadcast to chain (pending confirmation)
              └──────────────┘
                    │
          ┌─────────┴─────────┐
          │                   │
          ▼                   ▼
   ┌──────────────┐     ┌──────────────┐
   │ CONFIRMED    │     │   FAILED     │  Tx failed or timed out
   └──────────────┘     └──────────────┘
          │                   │ retry → SUBMITTED
          │                   │
          ▼                   ▼
   ┌──────────────┐     ┌──────────────┐
   │ REVERIFIED   │     │ SUBMITTED    │  (retry loop)
   └──────────────┘     └──────────────┘
```

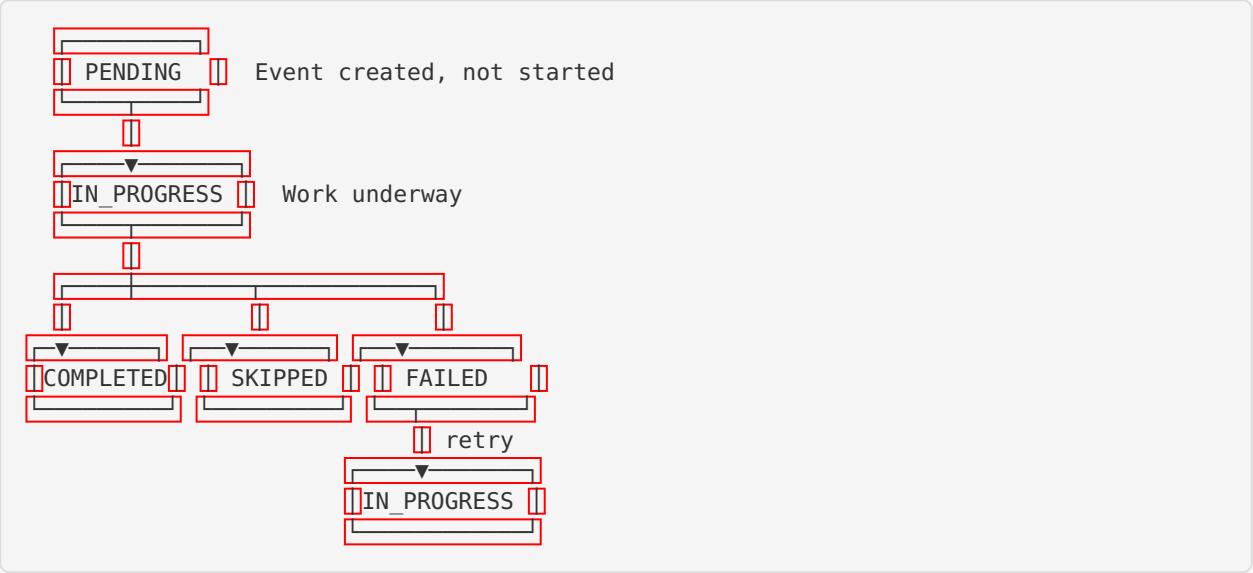
State transition rules:

From	To	Trigger
DRAFT	HASHED	`hashAttestation()` — computes hash and signature
HASHED	SUBMITTED	`submitAnchor()` — sends to blockchain
SUBMITTED	CONFIRMED	Blockchain confirmation received
SUBMITTED	FAILED	Tx failure, timeout, or chain error

FAILED	SUBMITTED	Retry submission
CONFIRMED	REVERIFIED	`reverify()` — re-checked and still valid
HASHED	HASHED	Can remain hashed indefinitely if no blockchain

Note: An attestation at HASHED is already fully valid for local provenance. Blockchain anchoring is optional.

5.2 Pathway Event Lifecycle



Pathway Event can also → CANCELLED from any non-terminal state.

From	To	Condition
PENDING	IN_PROGRESS	Stage work begins
IN_PROGRESS	COMPLETED	Stage requirements met
IN_PROGRESS	SKIPPED	Clinical decision to skip
IN_PROGRESS	FAILED	Stage could not be completed
IN_PROGRESS	CANCELLED	Episode withdrawn
PENDING	CANCELLED	Episode withdrawn
PENDING	SKIPPED	Pre-determined skip
FAILED	IN_PROGRESS	Retry

5.3 Approval Lifecycle

```
┌─────────┐
│ PENDING │  Approval requested
└─────────┘
     │
  ┌──┴──┐
  │     │
  ▼     ▼
┌────────┐ ┌────────┐
│APPROVED│ │REJECTED│
└────────┘ └────────┘

(any non-terminal) → WITHDRAWN (by requester)
```

From	To	Actor
PENDING	APPROVED	Reviewer (Admin/Coordinator)
PENDING	REJECTED	Reviewer (Admin/Coordinator)
PENDING	WITHDRAWN	Original requester

5.4 Import Lifecycle

```
┌─────────┐
│ PENDING │  File uploaded, import not started
└─────────┘
     │ startImport()
     ▼
┌──────────┐
│VALIDATING│  Schema and format checks
└──────────┘
     │
  ┌──┴──┐
  │     │
  ▼     ▼
┌─────────┐ ┌──────┐
│VALIDATED│ │FAILED│  Validation errors
└─────────┘ └──────┘
     │ executeImport()
     ▼
┌─────────┐
│IMPORTING│  Row-by-row processing
└─────────┘
     │
  ┌──┴──┐
  │     │
  ▼     ▼
┌─────────┐ ┌────────────────┐
│COMPLETED│ │PARTIALLY_COMPLETED│  Some rows had errors
└─────────┘ └────────────────┘
```

6. Error Cases & Retry Behavior

6.1 Blockchain Unavailable (Graceful Degradation)

Scenario	Behavior
RPC endpoint unreachable	`BlockchainService.isAvailable()` returns false; `submitAnchor()` returns `{ success: false, error: 'CHAIN_UNAVAILABLE' }`
Attestation remains at HASHED	Fully valid for local provenance. Queue for later anchoring.
UI indication	Show “Not anchored — local attestation valid” badge
Retry	Background job retries queued anchors every 5 minutes, max 10 retries with exponential backoff
After max retries	Status moves to FAILED. Manual retry available.

Retry strategy for blockchain submissions:

```
Attempt 1: immediate
Attempt 2: 30 seconds
Attempt 3: 2 minutes
Attempt 4: 5 minutes
Attempt 5-10: 5 minutes each
After attempt 10: FAILED, manual intervention required
```

6.2 Hash Mismatch on Verification

Scenario	Detection	Response
Payload hash doesn’t match stored hash	`verifyAttestation()` recomputes hash from `payloadCanonical`	Return `PAYLOAD_TAMPERED` status; log audit entry with severity HIGH
Signature doesn’t verify	HMAC recomputed with current key	Return `SIGNATURE_MISMATCH` status; log audit entry
On-chain hash doesn’t match	Query contract event logs	Return `ANCHOR_MISMATCH` status; flag for investigation

Response protocol: All mismatches are logged as audit entries with full before/after state. No automatic correction — human review required.

6.3 Import Validation Failures

Error Type	Handling
Invalid file format	Reject at VALIDATING, set FAILED with error message
Missing required columns	Reject at VALIDATING, list missing columns in `errors`
Invalid data values	Per-row error tracking in `errors` JSON array
Duplicate pseudo-IDs	Warning + skip or error based on import config
Partial success	Set PARTIALLY_COMPLETED, report `processedRows` vs `errorRows`

Error format in `ImportJob.errors`:

```
interface ImportError {
  row?: number;
  field?: string;
  code: string;       // e.g. "MISSING_REQUIRED", "INVALID_FORMAT", "DUPLICATE"
  message: string;
  severity: 'error' | 'warning';
}
```

6.4 Concurrent Modification Conflicts

Resource	Strategy
Patient episodes	Optimistic concurrency via `updatedAt` check. If stale, reject with 409 Conflict.
Pathway events	Lock-based for status transitions. `Lock` table prevents concurrent modifications.
Attestations	Immutable once HASHED. No concurrent modification possible.
Checkpoints	Append-only. Version number ensures ordering.
Projects	Optimistic concurrency via `updatedAt`.

Lock behavior:
- Locks auto-expire after 30 minutes (configurable).
- Lock acquisition fails with 423 Locked if already held by another user.
- Admin can force-release locks.

6.5 General Error Handling Pattern

All service functions follow this pattern:

```
class ServiceError extends Error {
  constructor(
    public code: string,          // e.g. "ATTESTATION_NOT_FOUND"
    public statusCode: number,    // HTTP status code
    message: string,
    public details?: Record<string, unknown>
  ) {
    super(message);
  }
}

// Standard error codes:
// 400 - BAD_REQUEST, VALIDATION_FAILED, INVALID_STATE_TRANSITION
// 401 - UNAUTHORIZED
// 403 - FORBIDDEN, INSUFFICIENT_ROLE
// 404 - NOT_FOUND
// 409 - CONFLICT, CONCURRENT_MODIFICATION
// 423 - LOCKED
// 500 - INTERNAL_ERROR
// 502 - CHAIN_UNAVAILABLE
// 503 - SERVICE_UNAVAILABLE
```

---

7. Chain-Agnostic vs Chain-Specific Separation

7.1 Separation Boundary

```
╔═══════════════════════════════════════════════════════════╗
║                    CHAIN-AGNOSTIC LAYER                   ╒
│  (src/lib/services/attestation.service.ts)               │
│  (src/lib/services/blockchain.service.ts)                │
│                                                          │
│  • AttestationPayload creation and serialization         │
│  • Canonical JSON deterministic serialization            │
│  • SHA-256 hashing                                       │
│  • HMAC signing / signature verification                 │
│  • Attestation lifecycle state machine                   │
│  • Verification logic (integrity + signature)            │
│  • BlockchainProvider interface definition               │
│  • Provider registry and selection                       │
│  • Retry/queue logic for anchor submissions              │
│  • Graceful degradation when chain unavailable           │
╘                                                          ╛
╔═══════════════════════════════════════════════════════════╗
║                    CHAIN-SPECIFIC LAYER                   ╒
│  (src/lib/services/blockchain/base.provider.ts)          │
│  (src/lib/services/blockchain/noop.provider.ts)          │
│  (future: optimism.provider.ts, polygon.provider.ts)     │
│                                                          │
│  • Contract ABI definitions                              │
│  • RPC endpoint configuration                            │
│  • Gas estimation and tx construction                    │
│  • Transaction submission via ethers.js                  │
│  • Event log parsing for verification                    │
│  • Chain-specific error handling                         │
│  • Confirmation waiting logic                            │
│  • Provider-specific configuration (gas limits, etc.)    │
╘                                                          ╛
╚═══════════════════════════════════════════════════════════╝
```

7.2 Adding a New Chain

To add support for a new chain, implement `BlockchainProvider`:

```
// src/lib/services/blockchain/optimism.provider.ts
export class OptimismProvider implements BlockchainProvider {
  readonly chainId = 'optimism';
  // ... implement submitAnchor, verifyAnchor, getAnchorStatus, isAvailable
}
```

Then register it:

```
blockchainService.registerProvider(new OptimismProvider(config));
```

No changes to attestation logic, verification, or lifecycle management needed.

7.3 Canonical Serialization Specification

Deterministic JSON serialization ensures identical payloads always produce the same hash:

1. **Sort all object keys alphabetically** (recursive)

2. **No whitespace** (`JSON.stringify` with no indent)
3. **Dates as ISO 8601 strings** (UTC, millisecond precision)
4. **Numbers as-is** (no scientific notation normalization for now)
5. **Null values included** (not stripped)
6. **Undefined values excluded**

Implementation:

```
function canonicalize(obj: Record<string, unknown>): string {
  return JSON.stringify(obj, Object.keys(obj).sort(), 0);
  // Note: recursive sort requires a custom replacer — full implementation
  // uses a deep-sort utility before JSON.stringify.
}
```

---

8. API Routes

All routes are under `/api/`. Authentication is via `Authorization: Bearer <jwt>` header.

8.1 Auth Routes

Method	Path	Description	Auth
POST	`/api/auth/register`	Register new user	None (or Admin)
POST	`/api/auth/login`	Login, receive JWT	None
POST	`/api/auth/logout`	Revoke current session	Required
GET	`/api/auth/me`	Get current user	Required
PUT	`/api/auth/password`	Change password	Required

8.2 Project Routes

Method	Path	Description	Roles
POST	`/api/projects`	Create project	Admin, Researcher
GET	`/api/projects`	List projects (filtered by membership)	All
GET	`/api/projects/:id`	Get project details	Members
PUT	`/api/projects/:id`	Update project	Admin, Coordinator
PUT	`/api/projects/:id/status`	Change project status	Admin
POST	`/api/projects/:id/sites`	Add site	Admin, Coordinator
GET	`/api/projects/:id/sites`	List sites	Members
PUT	`/api/projects/:id/sites/:siteId`	Update site	Admin, Coordinator
POST	`/api/projects/:id/members`	Add member	Admin
GET	`/api/projects/:id/members`	List members	Members
DELETE	`/api/projects/:id/members/:userId`	Remove member	Admin

8.3 Patient Episode Routes

Method	Path	Description	Roles
POST	`/api/projects/:projectId/episodes`	Create episode	Researcher, Coordinator
GET	`/api/projects/:projectId/episodes`	List episodes	Members
GET	`/api/projects/:projectId/episodes/:id`	Get episode detail	Members
PUT	`/api/projects/:projectId/episodes/:id`	Update episode	Researcher, Coordinator

8.4 Pathway Routes

Method	Path	Description	Roles
GET	`/api/pathway/stages`	List stage definitions	All
POST	`/api/episodes/:episodeId/events`	Create pathway event	Researcher, Coordinator
GET	`/api/episodes/:episodeId/events`	Get episode timeline	Members
PUT	`/api/events/:eventId`	Update event data	Researcher, Coordinator
PUT	`/api/events/:eventId/status`	Advance event status	Researcher, Coordinator
GET	`/api/episodes/:episodeId/progress`	Get pathway progress	Members

8.5 FHIR Resource Routes

Method	Path	Description	Roles
POST	`/api/episodes/:episodeId/fhir`	Create FHIR resource	Researcher, Coordinator
GET	`/api/episodes/:episodeId/fhir`	List FHIR resources	Members
GET	`/api/fhir/:id`	Get FHIR resource	Members
PUT	`/api/fhir/:id`	Update FHIR resource (creates new version)	Researcher, Coordinator

8.6 Import Routes

Method	Path	Description	Roles
POST	`/api/projects/:projectId/import/upload`	Upload file	Researcher, Coordinator
POST	`/api/import/:jobId/start`	Start import	Researcher, Coordinator
GET	`/api/import/:jobId`	Get import status	Members
GET	`/api/projects/:projectId/imports`	List import jobs	Members
GET	`/api/projects/:projectId/artifacts/input`	List input artifacts	Members

8.7 Attestation Routes

Method	Path	Description	Roles
POST	`/api/projects/:projectId/attestations`	Create attestation	Researcher, Coordinator
POST	`/api/attestations/:id/hash`	Hash attestation	Researcher, Coordinator
POST	`/api/attestations/:id/anchor`	Submit to blockchain	Admin, Coordinator
POST	`/api/attestations/:id/verify`	Verify attestation	All
GET	`/api/attestations/:id`	Get attestation	Members
GET	`/api/projects/:projectId/attestations`	List attestations	Members

8.8 Provenance Routes

Method	Path	Description	Roles
GET	`/api/provenance/:entityType/:entityId`	Get provenance record	Members
GET	`/api/provenance/:entityType/:entityId/chain`	Get full lineage chain	Members
GET	`/api/provenance/:entityType/:entityId/verify`	Verify chain integrity	All

8.9 Export Routes

Method	Path	Description	Roles
POST	`/api/projects/:projectId/export/fhir`	Export FHIR bundle	Researcher, Coordinator
POST	`/api/projects/:projectId/export/csv`	Export CSV report	Researcher, Coordinator
POST	`/api/projects/:projectId/export/package`	Export full package	Admin, Coordinator
GET	`/api/projects/:projectId/artifacts/output`	List output artifacts	Members
GET	`/api/artifacts/output/:id/download`	Download output artifact	Members

8.10 Checkpoint Routes

Method	Path	Description	Roles
POST	`/api/projects/:projectId/checkpoints`	Create checkpoint	Admin, Coordinator
GET	`/api/projects/:projectId/checkpoints`	List checkpoints	Members
GET	`/api/checkpoints/:id`	Get checkpoint	Members
GET	`/api/checkpoints/:id1/diff/:id2`	Compare checkpoints	Members

8.11 Approval Routes

Method	Path	Description	Roles
POST	`/api/approvals`	Request approval	Researcher, Coordinator
PUT	`/api/approvals/:id`	Review (approve/reject)	Admin, Coordinator
PUT	`/api/approvals/:id/withdraw`	Withdraw approval	Requester
GET	`/api/approvals`	List approvals (filtered)	Members

8.12 Audit Routes

Method	Path	Description	Roles
GET	`/api/audit`	Search audit entries	Admin, Auditor
GET	`/api/audit/:entityType/:entityId`	Audit trail for entity	Admin, Auditor
GET	`/api/projects/:projectId/runs`	Run log for project	Members

8.13 Lock Routes

Method	Path	Description	Roles
POST	`/api/locks`	Acquire lock	Researcher, Coordinator
DELETE	`/api/locks/:id`	Release lock	Lock holder, Admin
GET	`/api/locks`	List active locks	Members

---

9. Security Model & RBAC

9.1 Role Definitions

Role	Scope	Description
ADMIN	Global + Project	Full system access. Manages users, projects, configurations. Can force-release locks, override approvals.
RESEARCHER	Project	Primary data worker. Creates episodes, records pathway events, imports data, creates attestations.
COORDINATOR	Project	Clinical trial coordinator. Similar to Researcher but can also manage project settings, approve outputs, manage sites.
AUDITOR	Project (read)	Read-only plus audit access. Can view all data, provenance, audit trails. Cannot modify clinical data.

9.2 RBAC Permissions Matrix

Operation	Admin	Researcher	Coordinator	Auditor
Users				
Create user	✅	❌	❌	❌
Manage users	✅	❌	❌	❌
Projects				
Create project	✅	✅	❌	❌
Update project	✅	❌	✅	❌
Change project status	✅	❌	❌	❌
Delete project	✅	❌	❌	❌
View project	✅	✅	✅	✅
Project Members				
Add/remove members	✅	❌	❌	❌
View members	✅	✅	✅	✅
Sites				
Create/update site	✅	❌	✅	❌
View sites	✅	✅	✅	✅
Patient Episodes				
Create episode	✅	✅	✅	❌
Update episode	✅	✅	✅	❌
View episode	✅	✅	✅	✅
Pathway Events				
Create event	✅	✅	✅	❌

Operation	Admin	Researcher	Coordinator	Auditor
Update event status	✅	✅	✅	❌
View events	✅	✅	✅	✅
FHIR Resources				
Create/update	✅	✅	✅	❌
View	✅	✅	✅	✅
Import				
Upload & import	✅	✅	✅	❌
View imports	✅	✅	✅	✅
Attestations				
Create & hash	✅	✅	✅	❌
Submit to blockchain	✅	❌	✅	❌
Verify	✅	✅	✅	✅
View	✅	✅	✅	✅
Provenance				
View provenance	✅	✅	✅	✅
Verify integrity	✅	✅	✅	✅
Export				
Export data	✅	✅	✅	❌
Export full package	✅	❌	✅	❌
View/download outputs	✅	✅	✅	✅
Checkpoints				

Operation	Admin	Researcher	Coordinator	Auditor
Create checkpoint	✅	❌	✅	❌
View/compare	✅	✅	✅	✅
Approvals				
Request approval	✅	✅	✅	❌
Approve/reject	✅	❌	✅	❌
View approvals	✅	✅	✅	✅
Locks				
Acquire lock	✅	✅	✅	❌
Release own lock	✅	✅	✅	❌
Force-release any lock	✅	❌	❌	❌
Audit				
View audit log	✅	❌	❌	✅
View run logs	✅	✅	✅	✅

9.3 Authentication Flow

```
1. POST /api/auth/login { email, password }
2. Server validates credentials (bcrypt compare)
3. Server creates Session record (expiry: 24h default)
4. Server returns JWT containing { userId, sessionId, role }
5. Client includes JWT in Authorization: Bearer header
6. Middleware validates JWT, checks session is ACTIVE and not expired
7. AuthContext { user, session, projectRole? } attached to request
```

9.4 Project-Level Role Resolution

Users have a global `role` on the `User` model and a project-specific `role` on `ProjectMember`. The **effective role** for a project operation is:

1. If user is global ADMIN → always ADMIN
2. Else → use `ProjectMember.role` for that project
3. If no `ProjectMember` record → access denied (403)

9.5 Security Constraints

Constraint	Implementation
Passwords	bcrypt, min 12 rounds
JWT	HS256, 24h expiry, session-bound
Rate limiting	100 req/min per IP (auth endpoints: 10/min)
Input validation	Zod schemas on all API inputs
SQL injection	Prisma parameterized queries (ORM-level protection)
XSS	React auto-escaping + CSP headers
CSRF	SameSite cookies + CSRF token for state-changing ops
PHI	No real patient identifiers stored. Only pseudonymized IDs.
Audit	All state changes logged with actor, timestamp, before/after
Blockchain keys	Server-side only. No exposure to frontend.
HMAC secret	Environment variable, never in code or DB

---

10. Appendix: FHIR Mapping Reference

10.1 Pathway Stage → FHIR Resource Mapping

Pathway Stage	Primary FHIR Resources	Typical Observations
Symptom Screening	QuestionnaireResponse	Gait score, cognitive assessment, urinary symptoms
Imaging	Observation, DocumentReference	Evans index, callosal angle, DESH pattern, ventricular volume
Specialist Review	ServiceRequest, CarePlan	Referral, assessment notes
CSF Testing	Observation, ServiceRequest	Opening pressure, tap test response, drainage volume
Treatment Decision	CarePlan	Decision rationale, planned procedure
Shunt Intervention	ServiceRequest, DocumentReference	Procedure record, device details
Follow-Up	QuestionnaireResponse, Observation	Gait/cognition/urinary at 3mo, 6mo, 12mo

10.2 Observation Code System

For iNPH-specific observations, we use a local code system `http://nph-trust.local/fhir/CodeSystem/inph`:

Code	Display	Unit	Stage
`evans-index`	Evans Index	ratio	Imaging
`callosal-angle`	Callosal Angle	degrees	Imaging
`desh-pattern`	DESH Pattern	code	Imaging
`ventricular-volume`	Ventricular Volume	mL	Imaging
`csf-opening-pressure`	CSF Opening Pressure	cmH2O	CSF Testing
`tap-test-response`	Tap Test Response	code	CSF Testing
`gait-score`	Gait Score	score	Screening, Follow-Up
`cognitive-score`	Cognitive Score	score	Screening, Follow-Up
`urinary-score`	Urinary Symptom Score	score	Screening, Follow-Up
`modified-rankin`	Modified Rankin Scale	score	Follow-Up

10.3 File Structure (Planned)

```
src/
├── app/                          # Next.js App Router
│   ├── api/                      # API routes
│   │   ├── auth/
│   │   ├── projects/
│   │   ├── episodes/
│   │   ├── events/
│   │   ├── fhir/
│   │   ├── import/
│   │   ├── attestations/
│   │   ├── provenance/
│   │   ├── export/
│   │   ├── checkpoints/
│   │   ├── approvals/
│   │   ├── locks/
│   │   └── audit/
│   ├── (dashboard)/              # Dashboard pages
│   ├── (auth)/                   # Auth pages
│   └── layout.tsx
├── lib/
│   ├── services/                 # Service layer
│   │   ├── auth.service.ts
│   │   ├── project.service.ts
│   │   ├── pathway.service.ts
│   │   ├── attestation.service.ts
│   │   ├── provenance.service.ts
│   │   ├── import.service.ts
│   │   ├── export.service.ts
│   │   ├── blockchain.service.ts
│   │   ├── checkpoint.service.ts
│   │   ├── audit.service.ts
│   │   └── blockchain/
│   │       ├── provider.interface.ts
│   │       ├── base.provider.ts
│   │       └── noop.provider.ts
│   ├── types/                    # TypeScript interfaces
│   │   ├── auth.types.ts
│   │   ├── project.types.ts
│   │   ├── pathway.types.ts
│   │   ├── fhir.types.ts
│   │   ├── attestation.types.ts
│   │   ├── provenance.types.ts
│   │   ├── artifact.types.ts
│   │   ├── checkpoint.types.ts
│   │   └── common.types.ts
│   ├── utils/
│   │   ├── canonical.ts          # Deterministic JSON serialization
│   │   ├── hash.ts               # SHA-256 utilities
│   │   ├── errors.ts             # ServiceError class
│   │   └── validation.ts         # Zod schemas
│   ├── middleware/
│   │   ├── auth.middleware.ts
│   │   └── rbac.middleware.ts
│   └── db/
│       └── prisma.ts             # Prisma client singleton
├── prisma/
│   ├── schema.prisma
│   └── seed.ts                   # Seed pathway stage definitions
└── ...
```

End of Document

> This document is the authoritative design reference for NPH-Trust App 1. All implementation must conform to the schemas, interfaces, service boundaries, lifecycle states, error handling, and security model defined herein. Deviations require updating this document first.